

DoConnect AI

Intelligent Discussion & Knowledge Collaboration Platform

Description:

DoConnect AI is a full-stack web-based discussion platform designed to enable users to share knowledge by asking questions, providing answers, and receiving real-time notifications. The system follows a structured architecture similar to enterprise-level applications and supports role-based access control for secure usage.

Project Overview

This project is designed to simulate a real-world Q&A platform like Stack Overflow. It allows users to interact through questions and answers while maintaining proper authentication, user tracking, and notification systems. The platform ensures secure communication using JWT-based authentication and role-based dashboards.

Objectives

The main objective of this system is to create a collaborative environment where users can share technical knowledge efficiently. It focuses on secure login, structured data management, and real-time interaction between users through notifications.

- Enable knowledge sharing
- Provide secure authentication
- Maintain user activity tracking
- Improve communication through notifications

Technologies Used:

The application is developed using modern web technologies to ensure scalability, performance, and security.

Frontend:

- React JS → UI development
- Axios → API communication
- Bootstrap → styling
- Local Storage → session management

Backend:

- Spring Boot → backend framework
- Spring MVC → layered architecture
- Spring Security → authentication
- JWT → token-based security
- Spring Data JPA → database operations

Database:

- MySQL → persistent data storage

System Architecture:

The system follows a layered architecture pattern where each layer has a specific responsibility. This ensures separation of concerns, maintainability, and scalability.

Frontend (React)



Controller Layer (REST APIs)



Service Layer (Business Logic)



Repository Layer (Database Access)



MySQL Database

Additionally:

- Entity Layer → defines database structure
- DTO Layer → handles request/response objects
- Security Layer → manages authentication and authorization

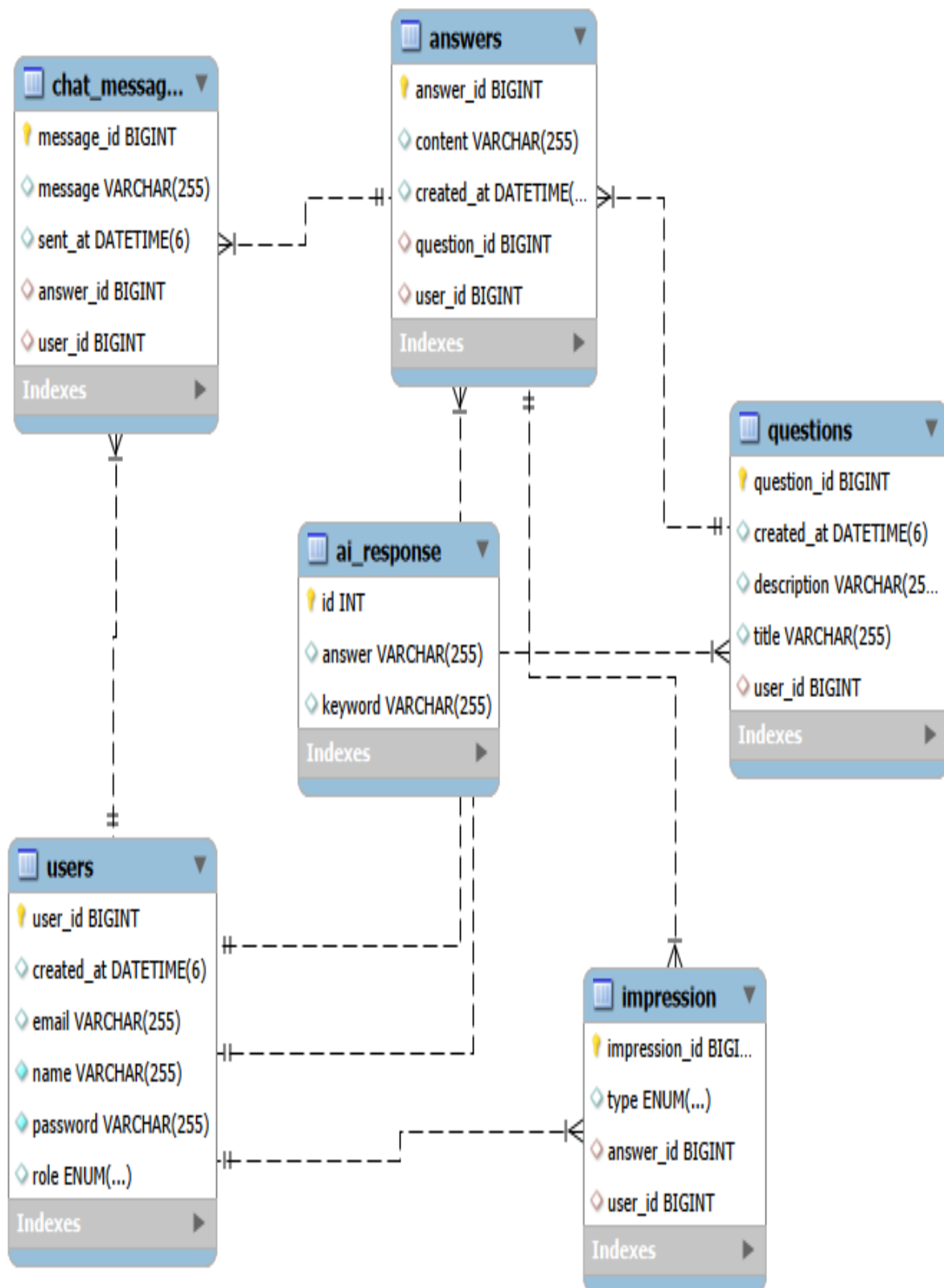
ER Diagram:

An Entity Relationship (ER) Diagram is a visual representation of the database structure that shows how different entities (tables) are connected to each other. It helps in understanding the relationships between data objects and ensures proper database design with minimal redundancy and maximum consistency.

The main entities in the system are:

- User
- Question
- Answer
- Impression
- ChatMessage
- AIResponse

Each entity contains attributes that define its properties, and relationships are established using primary and foreign keys.



Relationship Description

1. **User** ↔ **Question** (1 : Many Relationship)

A User can post multiple Questions, but each Question is posted by only one User.

- This is a one-to-many relationship
- Implemented using `user_id` as a foreign key in the Question table
- It ensures that every question can be traced back to the user who created it.

One user can actively participate by posting multiple questions, while each question belongs to a single user.

2. **Question** ↔ **Answer** (1 : Many Relationship)

A Question can have multiple Answers, but each Answer belongs to only one Question.

- This is a one-to-many relationship
- Implemented using `question_id` as a foreign key in the Answer table
- It ensures that all answers are linked to their respective questions

A single question can receive responses from multiple users, enabling collaborative knowledge sharing.

3. **User** ↔ **Answer** (1 : Many Relationship)

A User can submit multiple Answers, but each Answer is posted by only one User.

- Implemented using `user_id` as a foreign key in the Answer table
- Helps track answer ownership and user contribution

Users can contribute knowledge by answering multiple questions across the platform.

4. **Answer** ↔ **Impression** (1 : Many Relationship)

An Answer can receive multiple Impressions (Likes or Dislikes), but each Impression belongs to one Answer.

- Implemented using answer_id as a foreign key in the Impression table
- Supports user feedback and answer popularity tracking

Users can express their opinion about answers through likes and dislikes.

5. **User** ↔ **Impression** (1 : Many Relationship)

A User can create multiple Impressions, but each Impression belongs to one User.

- Implemented using user_id as a foreign key
- Helps track which user liked or disliked an answer

Each user can provide feedback on multiple answers.

6. **Answer** ↔ **ChatMessage** (1 : Many Relationship)

An Answer can contain multiple Chat Messages, but each Chat Message belongs to one Answer.

- Implemented using answer_id as a foreign key in ChatMessage.

Users can discuss or communicate further regarding a particular answer.

7. **User** ↔ **ChatMessage** (1 : Many Relationship)

A User can send multiple Chat Messages, but each Chat Message is sent by one User.

- Implemented using user_id as a foreign key.

Users can participate in discussions by exchanging messages.

8. **AIResponse Entity**

The AIResponse entity stores AI-generated responses based on user-provided keywords.

Attributes:

- id
- keyword
- answer

Purpose:

This entity supports AI-assisted knowledge retrieval and automated answer generation.

UML Diagrams

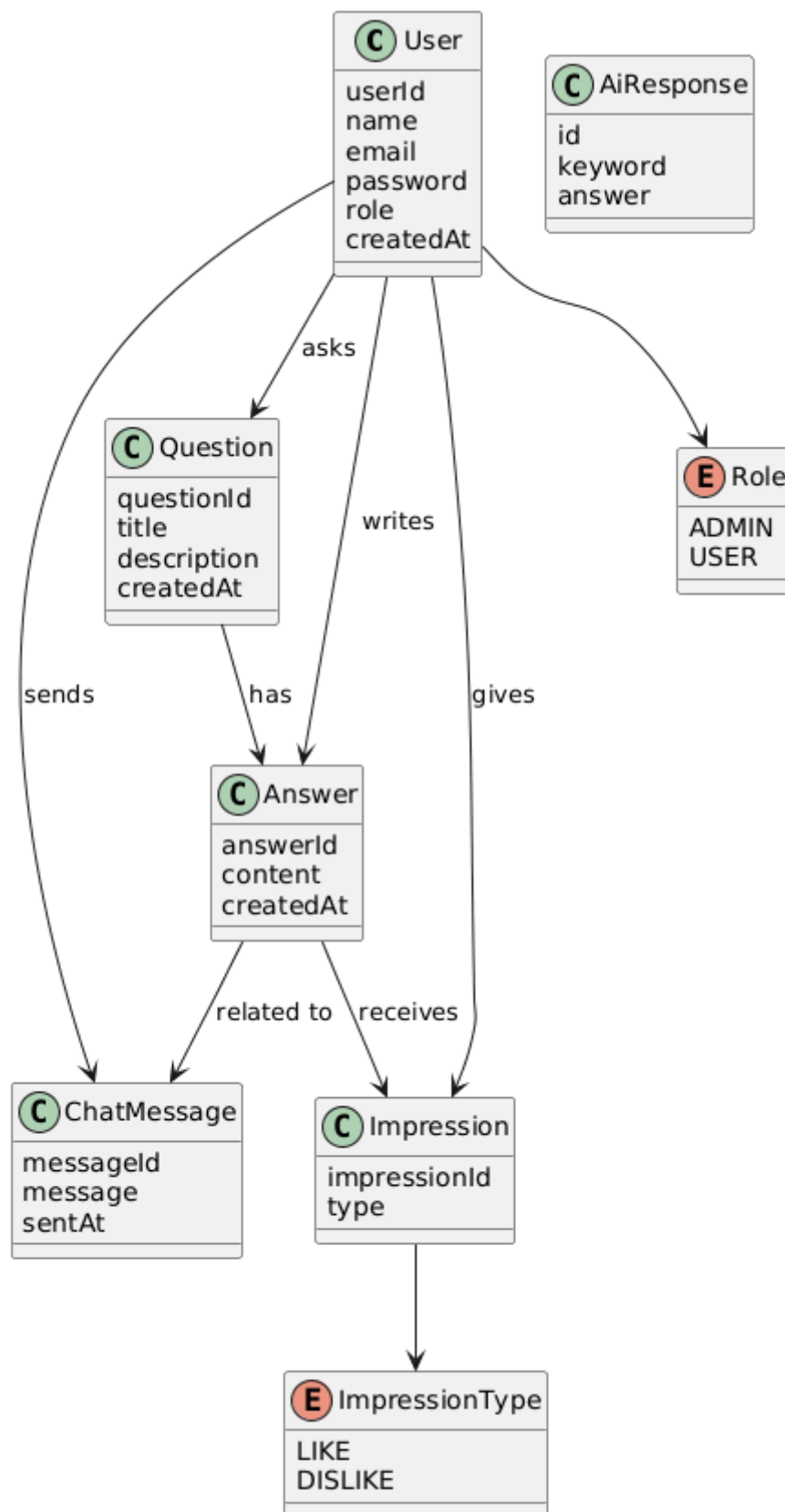
UML (Unified Modeling Language) is a standard visual language used to design, describe, and document software systems. It helps developers understand the structure and behavior of an application through graphical representations. UML diagrams provide a clear view of how different components interact with each other and help in planning the overall system architecture.

In the DoConnect platform, UML diagrams are used to represent the major entities such as User, Question, Answer, Impression, ChatMessage, and AIResponse. These diagrams illustrate the attributes of each class and the relationships between them. By visualizing these connections, developers can better understand the flow of data and the responsibilities of each component within the system.

UML diagrams also improve communication among developers, testers, and stakeholders by providing a common representation of the application's design. They serve as an important part of system documentation and support future maintenance, scalability, and enhancements.

UML is mainly used to represent:

- System structure (classes and objects)
- System behavior and interactions
- Relationships between entities
- Data flow between components
- Overall application architecture



UML is mainly used to represent:

- Users and their activities
- Questions and Answers management
- Impression (Like/Dislike) system
- Chat messaging functionality
- AI-generated response management
- Relationships between different entities

Type of UML Diagram Used in Project

Your diagram is mainly a:

Class Diagram (Structural UML Diagram)

A Class Diagram shows:

- Classes (entities)
- Attributes (data fields)
- Relationships between classes

It represents the static structure of your system.

Explanation of Your Class Diagram

My system contains the following main classes:

1. User Class

Represents a system user.

Attributes:

- userId (Long)
- name (String)
- email (String)
- password (String)
- role (Role)
- createdAt (LocalDateTime)

Role in system:

- A user can ask questions
- A user can answer questions
- A user can send chat messages
- A user can like or dislike answers
- A user can access AI-generated responses

2. Question Class

Represents questions posted by users.

Attributes:

- questionId
- title
- description
- createdAt

Relationships:

- One User → Many Questions
- One Question → Many Answers (1 : Many)

3. Answer Class

Represents answers submitted for questions.

Attributes:

- answerId
- content
- createdAt

Relationships:

- One Question → Many Answers
- One User → Many Answers

- One Answer → Many Impressions
- One Answer → Many Chat Messages

4. Impression Class

Represents likes and dislikes given to answers.

Attributes:

- impressionId
- type (LIKE / DISLIKE)

Relationships:

- One User → Many Impressions
- One Answer → Many Impressions

Purpose:

Used to record user feedback on answers and identify valuable contributions within the platform.

5. ChatMessage Class

Represents chat messages related to answers.

Attributes:

- messageId
- message
- sentAt

Relationships:

- One User → Many Chat Messages
- One Answer → Many Chat Messages

Purpose:

Used to facilitate communication and discussion between users regarding answers.

6. AIResponse Class

Represents AI-generated responses stored in the system.

Attributes:

- id
- keyword
- answer

Purpose:

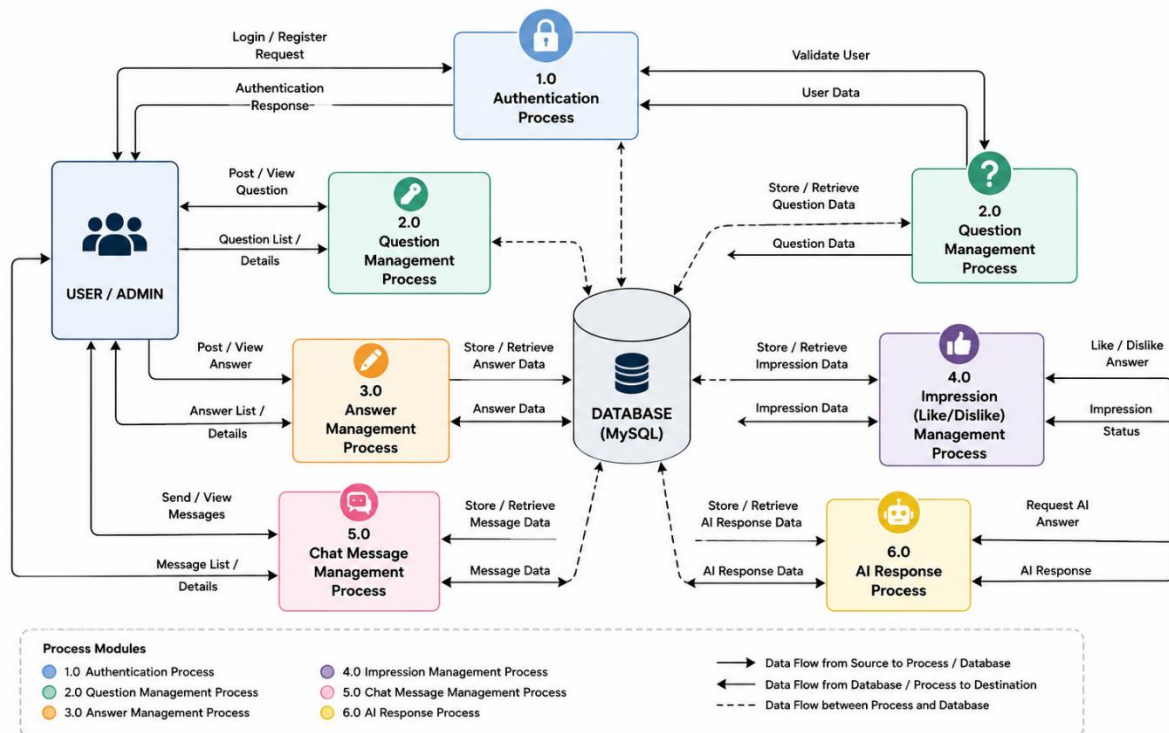
Stores AI-generated answers based on user-provided keywords and supports intelligent knowledge assistance within the platform.

Data Flow Diagrams:

A Data Flow Diagram (DFD) is a graphical representation that shows how data moves through a system. It describes the flow of data between external entities (users or systems), processes, and data storage (databases). DFD helps in understanding how input data is transformed into output through different system components.

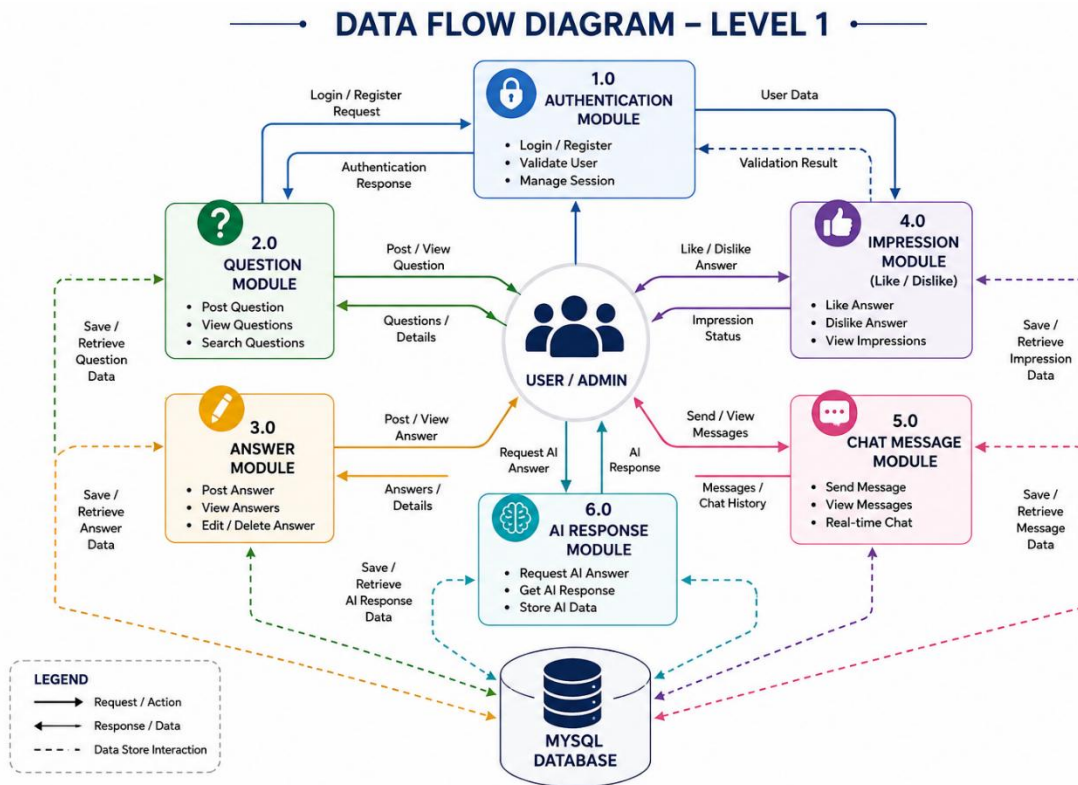
DFD does not focus on implementation details like code or hardware. Instead, it focuses on what data is moving, where it is coming from, where it is going, and how it is processed inside the system.

DATA FLOW DIAGRAM – LEVEL 1



Purpose of DFD

- To visualize system data flow clearly
- To understand system functionality at different levels
- To identify inputs, outputs, and data storage
- To help in system design and documentation



Module Description:

Authentication Module:

This module handles user registration and login functionality. It ensures that only valid users can access the system using JWT-based authentication.

Flow:

1. User registers or logs in
2. Backend validates credentials
3. JWT token is generated
4. Token + user details are returned
5. Frontend stores session data

User Management Module:

This module manages user-related operations such as storing user details, assigning roles, and tracking user identity across the system.

Question Management Module:

This module allows users to post technical questions and view questions posted by others. Each question is stored with metadata like username.

Data Stored:

- Title
- Description
- Likes
- PostedBy (username)

Answer Management Module:

This module enables users to provide answers to existing questions. Each answer is linked to a specific question and user.

Business Flow:

1. User submits answer
2. Backend fetches related question
3. Answer is saved in database

Impression Module:

This module is responsible for managing user feedback on answers through likes and dislikes. It helps identify useful answers and improves user engagement by allowing users to express their opinions on shared knowledge.

Flow:

1. User selects Like or Dislike on an answer
2. Backend validates the user and answer details
3. Impression type (LIKE/DISLIKE) is recorded
4. Impression data is stored in the database
5. Updated impression count and status are displayed in the frontend.

Admin Module:

The Admin Module provides administrative control over the platform. Admin users can monitor system activities, manage users, review questions and answers, and ensure the smooth functioning of the application. This module helps maintain data integrity and platform security.

Chat Message Module:

This module enables users to communicate through answer-based discussions. Users can send and view messages associated with specific answers, allowing better collaboration and interaction within the platform. All chat messages are stored and managed through the backend system.

AI Response Module:

This module integrates AI-generated responses to assist users in finding relevant information quickly. Based on user-provided keywords, the system generates intelligent responses and stores them for future reference. This feature enhances the knowledge-sharing experience and serves as an intelligent support mechanism within the platform.

Impression Module:

This module manages user feedback through Like and Dislike functionality. Users can express their opinions on answers, helping identify valuable contributions and improve the overall quality of discussions. Impression data is stored and tracked for each answer.

Backend Architecture:

The backend follows a structured Spring Boot layered architecture that separates responsibilities into different components for better scalability, maintainability, and security. Each layer performs a specific function and communicates with other layers in a well-defined manner, ensuring a clean and modular application design.

Controller Layer:

The Controller Layer handles incoming HTTP requests from the frontend and returns appropriate responses. It acts as the entry point of the backend system and maps user requests to the corresponding service methods.

Service Layer:

The Service Layer contains the core business logic of the application. It processes user requests, performs validations, manages interactions between different modules, and handles operations related to Questions, Answers, Impressions, Chat Messages, and AI Responses.

Repository Layer:

The Repository Layer is responsible for database operations using Spring Data JPA. It provides methods for storing, retrieving, updating, and deleting data from the MySQL database while abstracting database-specific implementation details.

Entity Layer:

The Entity Layer represents database tables as Java classes. It defines the application's data model and relationships between entities such as User, Question, Answer, Impression, ChatMessage, and AIResponse.

Security Layer (JWT):

The Security Layer provides authentication and authorization using Spring Security and JWT (JSON Web Tokens). It ensures that only authenticated users can access protected APIs by validating tokens for every request.

DTO Layer:

The DTO (Data Transfer Object) Layer is used to transfer data between the frontend and backend efficiently. It helps expose only the required information in API requests and responses while improving security and reducing unnecessary data transfer.

Frontend Architecture:

The frontend is built using React JS and follows a component-based architecture. Each feature is implemented as a reusable component, improving maintainability, scalability, and code reusability. The frontend provides an interactive user interface and communicates with the backend through REST APIs.

Pages:

- Login Page
- Register Page
- Dashboard Page
- Questions Page
- Add Question Page
- Answers Page
- Add Answer Page
- Chat Page
- User Profile Page
- Edit Profile Page
- Users Management Page

- Protected Route Component

State Management:

- React Hooks
- Component-based state management
- Local Storage for session handling

API Communication:

- Axios is used to communicate with backend APIs
- JWT token is attached to request headers for secure requests
- API services are managed through a centralized api.js service layer

Key Features:

The system includes several enterprise-level features that make it a complete full-stack knowledge-sharing platform.

- JWT Authentication
- Role-Based Access Control (Admin/User)
- Question Management System
- Answer Management System
- Impression System (Like/Dislike)
- Chat Messaging System
- User Profile Management
- AI Response Integration
- Protected Routes
- Timestamp Tracking
- User Activity Tracking
- Dashboard Management

Business Logic Highlights:

The backend automatically handles important operations such as user authentication, timestamp generation, linking questions with answers, managing impressions (likes/dislikes), storing chat messages, and generating AI-based responses without user intervention.

Security Implementation:

Security is implemented using Spring Security and JWT tokens. Every request is validated to ensure that only authenticated and authorized users can access protected resources. Role-based authorization is used to restrict access to specific functionalities based on user roles.

Future Enhancements:

The system is designed to be scalable and supports future improvements and advanced features.

- Enhanced AI-powered answer recommendations
- Real-time chat using WebSocket technology
- Advanced search and filtering functionality
- AI-based question categorization
- Analytics and reporting dashboard
- Email and push notifications
- Performance optimization and caching mechanisms

Conclusion

DoConnect AI is a complete full-stack application that demonstrates real-world enterprise development concepts such as authentication, layered architecture, notifications, and role-based access control. It showcases strong backend and frontend integration using modern technologies.

Submitted by:

Name: Srikar Akula

Role: Project Engineer

Email: srikar.akula@wipro.com

Under Guidance Of:

Trainer Name: Javeed Sir

Organization: Wipro

Date: 19-06-2026